

ui.page 全面详细解析

`ui.page` 是 NiceGUI 框架中核心的页面构建装饰器，用于将普通函数标记为页面生成器，为不同路由创建独立、用户私有的页面实例。每个访问对应路由的用户都会获得专属页面，不会与其他用户分享内容，极大地保障了页面的独立性和数据安全性。以下从核心特性、参数说明、关键功能用法、高级场景应用等方面进行全面解析。

一、核心特性

- 实例私有性：**每个用户访问路由时都会生成新的页面实例，页面状态、元素仅对当前用户可见，不存在多用户数据共享冲突。
- 路由全局注册：**通过 `path` 参数指定页面路由，路由需以 `/` 开头，注册后全局生效，用户可通过该路由访问对应页面。
- 函数类型限制：**仅支持修饰普通函数（free functions）和静态方法（static methods），不支持实例方法或初始化方法（需传入 `self` 参数，路由无法关联）。
- 灵活集成性：**可与 FastAPI 生态深度融合，支持 APIRouter 模块化、接收 FastAPI 请求对象等，适配复杂项目架构。

二、完整参数说明

`ui.page` 提供丰富的参数配置，满足页面个性化需求，各参数详情如下：

参数名	类型	说明	默认值
path	str	页面路由， 必须以 <code>/</code> 开头 （如 <code>/home</code> 、 <code>/user/{id}</code> ）	-（必填）
title	str	页面标题（显示在浏览器标签栏）	无
viewport	str	viewport 元标签内容（用于响应式布局控制，如 <code>width=device-width, initial-scale=1.0</code> ）	无
favicon	str	网站图标路径，支持相对路径（如 <code>./favicon.ico</code> ）或绝对 URL	None（使用 NiceGUI 默认图标）
dark	bool	是否启用 Quasar 深色模式	继承 <code>ui.run()</code> 命令的 <code>dark</code> 参数配置
language	str	页面语言（如 <code>zh-CN</code> 、 <code>en-US</code> ）	继承 <code>ui.run()</code> 命令的 <code>language</code> 参数配置
response_timeout	float	页面构建超时时间（装饰函数执行的最大允许时间）	3.0 秒
reconnect_timeout	float	服务器等待浏览器重连的最大时间	继承 <code>ui.run()</code> 命令的 <code>reconnect_timeout</code> 参数配置

参数名	类型	说明	默认值
api_router	APIRouter	用于注册页面的 APIRouter 实例（用于模块化开发）	None（使用默认路由）
kwargs	关键字参数	传递给 FastAPI <code>@app.get</code> 方法的额外参数（如 <code>tags</code> 、 <code>description</code> 等）	无

三、基础用法示例

1. 简单页面创建

通过 `@ui.page(path)` 装饰函数，快速创建多个独立页面，并通过 `ui.link` 实现页面跳转：

```
from nicegui import ui

# 首页（默认路由 '/'）
@ui.page('/')
def home_page():
    ui.label('欢迎访问首页')
    # 跳转到其他页面（通过函数引用或路由字符串指定目标）
    ui.link('访问其他页面', '/other_page')
    ui.link('访问深色模式页面', dark_page) # 直接引用页面函数

# 普通页面
@ui.page('/other_page', title='其他页面')
def other_page():
    ui.label('welcome to the other side')

# 深色模式页面
@ui.page('/dark_page', dark=True, favicon='./dark_icon.ico')
def dark_page():
    ui.label('welcome to the dark side')

ui.run()
```

2. 带路径参数的页面

支持在路由中定义参数（类似 FastAPI），并通过类型注解自动转换参数类型（bool、int、float、complex 等），还可接收 `request`（请求对象）和 `client`（客户端连接对象）参数：

```
from nicegui import ui
from fastapi import Request

# 路由参数自动类型转换
@ui.page('/repeat/{word}/{count}')
def repeat_page(word: str, count: int):
    ui.label(word * count) # 如访问 /repeat/Ho!/3, 显示 "Ho!Ho!Ho!"

# 接收 request 和 client 参数
@ui.page('/user/{user_id}')
```

```
def user_page(user_id: int, request: Request, client):
    ui.label(f'用户ID: {user_id}')
    ui.label(f'客户端IP: {request.client.host}')

@ui.page('/')
def home():
    ui.link('向圣诞老人问好', '/repeat/Ho!/3')
    ui.link('查看用户1001资料', '/user/1001')

ui.run()
```

四、高级功能用法

1. 等待客户端连接

通过在页面函数中添加 `client` 参数，并 `await client.connected()`，可等待服务器与浏览器的 WebSocket 连接建立后再执行后续代码。适用于需要运行 JavaScript、异步操作等依赖客户端连接的场景：

```
import asyncio
from nicegui import ui

@ui.page('/wait_for_connection')
async def wait_for_conn_page():
    # 立即显示的内容（无需等待连接）
    ui.label('页面加载中...')

    # 等待 WebSocket 连接建立
    await ui.context.client.connected()

    # 连接建立后执行的异步操作
    await asyncio.sleep(2)
    ui.label('2秒后显示的内容（已建立客户端连接）')

@ui.page('/')
def home():
    ui.link('测试客户端连接等待', wait_for_conn_page)

ui.run()
```

2. 多客户端广播 (Multicasting)

通过 `app.clients` 迭代器，可向指定页面的所有客户端发送更新（如通知、内容修改），适用于后台进程推送、跨页面消息同步等场景（需 NiceGUI 2.7.0+ 版本）：

```
from nicegui import app, ui

# 接收广播的页面
@ui.page('/multicast_receiver')
def receiver_page():
    ui.label('该页面将接收首页发送的消息')
```

```

# 广播消息函数
def send_broadcast(message: str):
    # 遍历访问 /multicast_receiver 路由的所有客户端
    for client in app.clients('/multicast_receiver'):
        with client: # 切换到客户端的元素上下文
            ui.notify(message) # 向客户端发送通知

# 发送广播的首页
@ui.page('/')
def home():
    ui.button('发送广播消息', on_click=lambda: send_broadcast('Hi, 所有接收者!'))
    # 新标签页打开接收页面
    ui.link('打开接收页面', '/multicast_receiver', new_tab=True)

ui.run()

```

3. 模块化开发 (APIRouter 集成)

通过 `APIRouter` 可将页面按功能分组，统一设置路由前缀，实现代码模块化（适合多文件、大型项目）。可将路由模块拆分到独立文件，再导入主程序：

主程序 (main.py)

```

from nicegui import APIRouter, app, ui

# 创建带前缀的路由
router = APIRouter(prefix='/sub-path')

# 路由下的页面（实际路由：/sub-path）
@router.page('/')
def sub_index():
    ui.label('这是 /sub-path 下的内容')

# 路由下的子页面（实际路由：/sub-path/sub-sub-path）
@router.page('/sub-sub-path')
def sub_sub_page():
    ui.label('这是 /sub-path/sub-sub-path 下的内容')

# 首页
@ui.page('/')
def home():
    ui.link('访问子路径页面', '/sub-path')
    ui.link('访问子子路径页面', '/sub-path/sub-sub-path')

# 注册路由到主应用
app.include_router(router)

ui.run()

```

多文件结构示例（推荐）

```
project/
├─ main.py          # 主程序（导入并注册路由）
├─ routes/
│   └─ sub_routes.py # 路由模块（存放 APIRouter 相关页面）
```

路由模块（routes/sub_routes.py）

```
from nicegui import APIRouter, ui

router = APIRouter(prefix='/sub-path')

@router.page('/')
def sub_index():
    ui.label('模块化路由下的首页')

@router.page('/detail')
def sub_detail():
    ui.label('模块化路由下的详情页')
```

主程序（main.py）

```
from nicegui import app, ui
from routes.sub_routes import router # 导入路由模块

@ui.page('/')
def home():
    ui.link('访问模块化路由', '/sub-path')

app.include_router(router) # 注册路由
ui.run()
```

五、注意事项

- 1. 页面函数名称无实际意义，可任意命名，路由由 `path` 参数决定。
- 2. 路径参数类型注解仅支持 `bool`、`int`、`float`、`complex`，未注解则默认保留字符串类型。
- 3. 等待客户端连接时，需确保页面函数为异步函数（添加 `async` 关键字），否则无法使用 `await`。
- 4. 多客户端广播时，需通过 `with client:` 切换到客户端上下文，否则无法修改该客户端的 UI 元素。
- 5. 模块化开发中，`APIRouter` 的 `prefix` 参数会自动添加到页面路由前，需注意路由拼接的正确性（如 `prefix='/sub' + page '/path'` 最终路由为 `/sub/path`）。

NiceGUI 中 ui.page 深度解析

在 NiceGUI 中，`ui.page` 是页面路由与组件挂载的核心装饰器，用于定义 Web 应用的页面路由规则、配置页面属性，并将 UI 组件挂载到对应的页面中。它是实现多页面应用的基础，封装了底层 FastAPI 的路由逻辑与前端页面的渲染机制，同时提供了灵活的页面配置、参数传递和生命周期管理能力。本文将从核心定位、基础用法、页面配置、参数传递、高级特性、常见问题六个维度，对 `ui.page` 进行全方位的详细阐述。

一、ui.page 的核心定位

ui.page 本质是一个路由装饰器，其核心定位可总结为三点：

- 1. **路由映射**：将 Python 函数与 URL 路径绑定，当用户访问该 URL 时，执行对应的函数并渲染页面；
- 2. **组件挂载**：函数内部通过 ui.* 系列组件创建的 UI 元素，会被自动挂载到当前页面的 DOM 中；
- 3. **页面配置**：支持通过参数自定义页面的标题、图标、响应式布局、依赖资源等属性。

ui.page 基于 FastAPI 的 @app.get 路由装饰器实现，同时扩展了前端页面的渲染逻辑，让开发者无需编写 HTML/CSS/JS，仅通过 Python 代码即可完成页面开发。

二、ui.page 的基础用法

ui.page 的基础使用遵循装饰器语法，核心是将一个函数装饰为页面处理函数，函数内部编写 UI 组件逻辑。

2.1 最简化的页面定义

```
from nicegui import ui

# 定义根路径页面：访问 http://localhost:8080/ 时触发
@ui.page('/')
def index_page():
    # 页面内的 UI 组件，自动挂载到页面中
    ui.label('Hello, NiceGUI!').classes('text-3xl font-bold')
    ui.button('点击我', on_click=lambda: ui.notify('按钮被点击了!'))

# 定义子路径页面：访问 http://localhost:8080/about 时触发
@ui.page('/about')
def about_page():
    ui.label('这是关于页面').classes('text-2xl')
    ui.link('返回首页', '/') # 页面跳转链接

if __name__ in {'__main__', '__mp_main__'}:
    ui.run(port=8080)
```

关键说明：

- 装饰器参数 / 和 /about 是页面的路由路径，支持绝对路径（以 / 开头）；
- 被装饰的函数（如 index_page）是页面处理函数，函数执行时会创建并渲染 UI 组件；
- 页面间跳转可通过 ui.link、ui.navigate.to() 实现，前者是静态链接，后者是动态跳转。

2.2 页面的默认行为

- **自动挂载**：页面处理函数内创建的所有 UI 组件，都会被挂载到当前页面的根容器中；
- **会话隔离**：每个页面访问都会关联到对应的 ui.session，不同客户端的页面状态相互隔离；
- **动态渲染**：页面处理函数在每次访问时都会执行，确保组件的最新状态被渲染。

三、ui.page 的页面配置参数

ui.page 提供了丰富的参数用于自定义页面属性，核心参数如下表所示：

参数名	类型	作用	示例
<code>path</code>	str	页面的路由路径（必传）	<code>@ui.page('/user')</code>
<code>title</code>	str	浏览器标签页标题	<code>@ui.page('/', title='首页')</code>
<code>icon</code>	str	浏览器标签页图标（支持 Font Awesome 图标）	<code>@ui.page('/', icon='home')</code>
<code>description</code>	str	页面的 meta 描述（SEO 用）	<code>@ui.page('/', description='NiceGUI 首页')</code>
<code>keywords</code>	str	页面的 meta 关键词（SEO 用）	<code>@ui.page('/', keywords='NiceGUI,Python,Web')</code>
<code>dark</code>	bool	是否强制启用深色模式	<code>@ui.page('/', dark=True)</code>
<code>lang</code>	str	页面的语言属性（如 zh-CN、en-US）	<code>@ui.page('/', lang='zh-CN')</code>
<code>head</code>	List[str]	向页面 <code><head></code> 中添加自定义标签	<code>@ui.page('/', head=[<meta name="author" content="NiceGUI">'])</code>
<code>on_connect</code>	Callable	页面连接时的回调函数	<code>@ui.page('/', on_connect=lambda: print('页面已连接'))</code>

3.1 配置页面标题、图标与深色模式

```
from nicegui import ui

# 配置首页的标题、图标和深色模式
@ui.page('/',
    title='我的 NiceGUI 应用',
    icon='star', # Font Awesome 图标
    dark=True, # 强制深色模式
    lang='zh-CN'
)

def index_page():
    ui.label('深色模式的首页').classes('text-3xl')

# 配置关于页的 SEO 信息
@ui.page('/about',
    description='这是一个基于 NiceGUI 的示例应用',
    keywords='NiceGUI,Python,Web开发'
)

def about_page():
    ui.label('关于页面 - SEO 配置示例')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

3.2 向页面头部添加自定义标签

适用于引入外部 CSS/JS 资源、添加自定义 meta 标签等场景：

```
from nicegui import ui

# 向页面 head 中添加外部 CSS 和 JS
custom_head = [
    # 引入外部 CSS (如 Bootstrap)
    '<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">',
    # 引入外部 JS
    '<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>',
    # 自定义 meta 标签
    '<meta name="viewport" content="width=device-width, initial-scale=1.0">'
]

@ui.page('/', head=custom_head)
def index_page():
    ui.label('引入外部资源的页面').classes('text-3xl')
    # 使用 Bootstrap 样式的按钮
    ui.button('Bootstrap 按钮').props('class="btn btn-primary"')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

四、ui.page 的参数传递

`ui.page` 支持两种参数传递方式：**路径参数**和**查询参数**，分别适用于不同的业务场景。

4.1 路径参数

路径参数是将参数嵌入到路由路径中，适用于**资源标识**（如用户 ID、文章 ID）等场景，语法为在路径中使用 `{参数名}`。

4.1.1 基础路径参数

```
from nicegui import ui

# 定义带路径参数的页面: {user_id} 是动态参数
@ui.page('/user/{user_id}')
def user_page(user_id: str): # 函数参数与路径参数同名
    ui.label(f'用户 ID: {user_id}').classes('text-2xl')
    ui.link('返回首页', '/')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```


访问 `http://localhost:8080/user/123` 时, 页面会显示 用户 ID: 123; 访问 `http://localhost:8080/user/456` 时, 显示 用户 ID: 456。

4.1.2 多路径参数与类型限制

支持多个路径参数, 且可通过 Python 类型注解限制参数类型 (如 `int`、`str`) :

```
from nicegui import ui

# 带多个路径参数的页面: 用户 ID (int) 和文章 ID (str)
@ui.page('/user/{user_id:int}/article/{article_id:str}')
def article_page(user_id: int, article_id: str):
    ui.label(f'用户 ID: {user_id} (整数类型)').classes('text-xl')
    ui.label(f'文章 ID: {article_id} (字符串类型)').classes('text-xl')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

说明: 若访问的参数类型不匹配 (如 `user_id` 传入字符串), NiceGUI 会返回 404 错误。

4.2 查询参数

查询参数是在 URL 后通过 `?key=value` 传递的参数, 适用于**筛选、分页、搜索**等场景, 通过 `ui.request` 获取。

```
from nicegui import ui

@ui.page('/search')
def search_page():
    # 通过 ui.request.query 获取查询参数
    query = ui.request.query.get('q', '默认搜索词') # 获取 q 参数, 无则返回默认值
    page = ui.request.query.get('page', '1') # 获取分页参数

    ui.label(f'搜索关键词: {query}').classes('text-2xl')
    ui.label(f'当前页码: {page}').classes('text-xl')

    # 生成带查询参数的链接
    ui.link('下一页', f'/search?q={query}&page={int(page)+1}')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

访问 `http://localhost:8080/search?q=nicegui&page=2` 时, 页面会显示搜索关键词 `nicegui` 和当前页码 `2`。

4.3 路径参数与查询参数结合

适用于更复杂的参数传递场景, 如 “用户详情页 + 筛选条件”:

```

from nicegui import ui

@ui.page('/user/{user_id:int}')
def user_detail_page(user_id: int):
    # 路径参数: 用户 ID
    ui.label(f'用户 ID: {user_id}').classes('text-2xl')
    # 查询参数: 筛选该用户的文章类型
    article_type = ui.request.query.get('type', 'all')
    ui.label(f'文章类型筛选: {article_type}').classes('text-xl')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

访问 `http://localhost:8080/user/123?type=tech` 时, 页面会显示用户 ID `123` 和文章类型 `tech`。

五、ui.page 的高级特性

5.1 页面的生命周期回调

`ui.page` 支持通过 `on_connect` 和 `on_disconnect` 配置页面的生命周期回调, 分别在客户端连接和断开页面时触发:

```

from nicegui import ui

# 页面连接时的回调
def on_page_connect():
    print(f'客户端 {ui.session.id} 连接到首页')
    # 初始化会话数据
    ui.session['visit_count'] = ui.session.get('visit_count', 0) + 1

# 页面断开时的回调
def on_page_disconnect():
    print(f'客户端 {ui.session.id} 断开了首页连接')

@ui.page('/', on_connect=on_page_connect, on_disconnect=on_page_disconnect)
def index_page():
    ui.label(f'当前会话访问次数: {ui.session.get("visit_count", 0)}').classes('text-2xl')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

5.2 嵌套页面与组件复用

通过将页面拆分为多个组件函数, 实现 UI 复用, 适用于多页面共享头部、尾部等布局:

```

from nicegui import ui

# 复用的头部组件
def page_header():
    with ui.header():
        ui.label('我的应用').classes('text-2xl font-bold')

```

```

        ui.link('首页', '/').classes('m1-4')
        ui.link('关于', '/about').classes('m1-2')

# 复用的尾部组件
def page_footer():
    with ui.footer():
        ui.label('@ 2025 NiceGUI 示例').classes('text-sm text-gray-500')

# 首页：使用复用组件
@ui.page('/')
def index_page():
    page_header() # 挂载头部
    with ui.main():
        ui.label('首页内容').classes('text-3xl')
    page_footer() # 挂载尾部

# 关于页：复用头部和尾部
@ui.page('/about')
def about_page():
    page_header() # 挂载头部
    with ui.main():
        ui.label('关于页内容').classes('text-3xl')
    page_footer() # 挂载尾部

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

5.3 动态注册页面

除了装饰器语法，NiceGUI 还支持通过 `ui.add_page` 动态注册页面，适用于**运行时根据配置生成页面**的场景：

```

from nicegui import ui

# 定义页面处理函数
def dynamic_page():
    ui.label('这是动态注册的页面! ').classes('text-3xl')
    ui.link('返回首页', '/')

# 动态注册页面：等价于 @ui.page('/dynamic')
ui.add_page('/dynamic', dynamic_page, title='动态页面')

# 首页
@ui.page('/')
def index_page():
    ui.label('首页').classes('text-3xl')
    ui.link('前往动态页面', '/dynamic')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

5.4 页面鉴权与拦截

结合装饰器和 `ui.session`，可实现对页面的访问权限控制，这是 `ui.page` 最常用的高级场景之一（前文鉴权案例的核心逻辑）：

```
from nicegui import ui

# 鉴权装饰器：检查登录状态
def require_login(page_func):
    def wrapper():
        if not ui.session.get('is_login', False):
            ui.notify('请先登录', type='error')
            return ui.navigate.to('/login')
        return page_func()
    return wrapper

# 登录页
@ui.page('/login')
def login_page():
    def do_login():
        ui.session['is_login'] = True
        ui.navigate.to('/protected')
    ui.button('模拟登录', on_click=do_login)

# 受保护页面：应用鉴权装饰器
@ui.page('/protected')
@require_login
def protected_page():
    ui.label('受保护的页面，仅登录后可访问').classes('text-3xl')
    def do_logout():
        ui.session['is_login'] = False
        ui.navigate.to('/login')
    ui.button('登出', on_click=do_logout)

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

5.5 异步页面处理

`ui.page` 支持异步页面处理函数，适用于需要执行异步操作（如数据库查询、网络请求）的场景：

```
from nicegui import ui
import asyncio

# 异步页面处理函数
@ui.page('/async')
async def async_page():
    ui.label('正在加载数据...').classes('text-2xl')
    # 模拟异步操作：如数据库查询、API 请求
    await asyncio.sleep(2)
    # 异步操作完成后更新页面
    ui.label('数据加载完成!').classes('text-2xl text-green-500')
```

```
if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

六、ui.page 的常见问题与解决方案

6.1 页面路由冲突

问题：定义多个相同路径的页面，导致路由冲突。

解决方案：确保每个页面的 `path` 参数唯一；若需覆盖原有页面，可先通过 `ui.remove_page(path)` 删除旧路由，再重新注册。

```
from nicegui import ui

# 初始首页
@ui.page('/')
def old_index():
    ui.label('旧首页')

# 删除旧路由并注册新首页
ui.remove_page('/')
@ui.page('/')
def new_index():
    ui.label('新首页')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

6.2 组件重复渲染

问题：页面处理函数内的组件在每次访问时重复创建，导致页面显示异常。

解决方案：

1. 确保组件创建逻辑在页面处理函数内，且仅执行一次；
2. 利用 `ui.session` 存储组件状态，避免重复渲染；
3. 使用 `ui.card()`、`ui.column()` 等容器组件管理组件层级。

6.3 路径参数类型错误

问题：访问带类型限制的路径参数时，传入错误类型导致 404 错误。

解决方案：

1. 在页面处理函数中对参数进行类型校验；
2. 使用 `try-except` 捕获类型转换异常；
3. 避免在路径参数中使用复杂类型，优先使用查询参数传递非基础类型数据。

6.4 页面跳转后状态丢失

问题：页面跳转后，`ui.session` 中的临时状态丢失。

解决方案：

1. 确认 `ui.session` 的使用在正确的上下文内；
2. 对于需要跨页面保留的状态，使用 `app.storage.user` 进行持久化；
3. 避免在页面处理函数外修改 `ui.session` 数据。

6.5 外部资源引入失败

问题：通过 `head` 参数引入的外部 CSS/JS 资源加载失败。

解决方案：

1. 检查资源 URL 是否正确，确保网络可访问；
2. 对于国内无法访问的资源，替换为国内 CDN 镜像；
3. 若资源需要异步加载，可通过 `ui.run_javascript()` 动态引入。

七、总结

`ui.page` 是 NiceGUI 实现多页面应用的**核心入口**，其装饰器语法简洁易懂，同时提供了丰富的配置参数和高级特性，满足从简单页面到复杂应用的开发需求。

核心要点回顾：

1. **基础用法：**通过装饰器绑定路由路径，函数内编写 UI 组件逻辑；
2. **页面配置：**支持标题、图标、深色模式、外部资源等自定义配置；
3. **参数传递：**通过路径参数传递资源标识，通过查询参数传递筛选条件；
4. **高级特性：**支持生命周期回调、组件复用、动态注册、鉴权拦截、异步处理；
5. **问题排查：**重点解决路由冲突、组件重复渲染、参数类型错误等常见问题。

掌握 `ui.page` 的使用，结合前文的 `ui.session` 和 `app` 类，可完整构建出功能完善的 NiceGUI 多页面应用，实现页面路由、状态管理、权限控制等核心业务需求。